

## **ASSIGNMENT-8.3**

**NAME:-PRATHAM KOUSHIK**

**ROLLNO:-2403A52182**

1. Use AI to generate test cases for `is_valid_email(email)` and then implement the validator function.

Requirements:

- Must contain @ and . characters.
- Must not start or end with special characters.
- Should not allow multiple @

## ▼ Generate test cases

### Subtask:

Generate a list of test cases covering valid and invalid email addresses based on

**Reasoning:** Initialize an empty list for test cases and add tuples of email strings and booleans based on the specified criteria.

```
[1] test_cases = []

# Valid email addresses
test_cases.append(("test@example.com", True))
test_cases.append(("firstname.lastname@example.co.uk", True))

# Email addresses missing the "@" symbol
test_cases.append(("testexample.com", False))

# Email addresses missing the "." symbol
test_cases.append(("test@examplecom", False))

# Email addresses starting with special characters
test_cases.append(("@test@example.com", False))
test_cases.append((".test@example.com", False))

# Email addresses ending with special characters
test_cases.append(("test@example.com.", False))
test_cases.append(("test@example.com@", False))

# Email addresses containing multiple "@" symbols
test_cases.append(("test@example@com", False))
test_cases.append(("test@@example.com", False))

# Edge cases
test_cases.append(("", False)) # Empty string
test_cases.append(("a@b.c", True)) # Short valid email
```

## ✓ Implement the function

### Subtask:

Implement the `is_valid_email` function that takes an email string as input and returns `True` if it is valid, `False` otherwise, based on the specified rules.

**Reasoning:** Implement the `is_valid_email` function according to the specified rules

```
def is_valid_email(email):  
    """  
    Validates an email string based on specific rules.  
  
    Args:  
        email: The email string to validate.  
  
    Returns:  
        True if the email is valid, False otherwise.  
    """  
    # Check if "@" is present  
    if "@" not in email:  
        return False  
  
    # Check if "." is present  
    if "." not in email:  
        return False  
  
    # Check if email starts with "@" or "."  
    if email.startswith("@") or email.startswith("."):   
        return False  
  
    # Check if email ends with "@" or "."  
    if email.endswith("@") or email.endswith("."):   
        return False  
  
    # Check for multiple "@" symbols  
    if email.count("@") != 1:  
        return False  
  
    # All checks passed, the email is valid  
    return True
```

## ✓ Test the function

### Subtask:

Use the generated test cases to test the `is_valid_email` function and verify its correctness.

**Reasoning:** Iterate through the test cases, call the `is_valid_email` function, and print the results.

```
3 for email, expected_result in test_cases:
    actual_result = is_valid_email(email)
    if actual_result == expected_result:
        print(f"Test passed for email: '{email}'. Expected: {expected_result}, Actual: {actual_result}")
    else:
        print(f"Test failed for email: '{email}'. Expected: {expected_result}, Actual: {actual_result}")
```

```
➡ Test passed for email: 'test@example.com'. Expected: True, Actual: True
Test passed for email: 'firstname.lastname@example.co.uk'. Expected: True, Actual: True
Test passed for email: 'testexample.com'. Expected: False, Actual: False
Test passed for email: 'test@examplecom'. Expected: False, Actual: False
Test passed for email: '@test@example.com'. Expected: False, Actual: False
Test passed for email: '.test@example.com'. Expected: False, Actual: False
Test passed for email: 'test@example.com.'. Expected: False, Actual: False
Test passed for email: 'test@example.com@'. Expected: False, Actual: False
Test passed for email: 'test@example@com'. Expected: False, Actual: False
Test passed for email: 'test@@example.com'. Expected: False, Actual: False
Test passed for email: ''. Expected: False, Actual: False
Test passed for email: 'a@b.c'. Expected: True, Actual: True
```

## ✓ Validate user input

### Subtask:

Prompt the user to enter an email address and validate it using the `is`

**Reasoning:** Prompt the user for input, call the `is_valid_email` function with the input, and print the validation result.

```
user_email = input("Enter an email address to validate: ")
if is_valid_email(user_email):
    print(f"'{user_email}' is a valid email address.")
else:
    print(f"'{user_email}' is not a valid email address.")
```

```
Enter an email address to validate: text@example
'text@example' is not a valid email address.
```

2 • Ask AI to generate test cases for `assign_grade(score)` function. Handle boundary and invalid inputs.

#### Requirements

- AI should generate test cases for `assign_grade(score)` where: 90-100: A, 80-89: B, 70-79: C, 60-69: D, <60: F
- Include boundary values and invalid inputs (e.g., -5, 105, "eighty")

## ✓ Generate test cases

### Subtask:

Generate a list of test cases covering valid scores within each grade range, boundary values, and invalid inputs (e.g., negative numbers, scores above 100, non-numeric inputs).

**Reasoning:** Initialize an empty list for test cases and add tuples of scores and expected grades based on the specified criteria.

```
[7] grade_test_cases = []

# Valid scores within each grade range
grade_test_cases.append((95, 'A'))
grade_test_cases.append((85, 'B'))
grade_test_cases.append((75, 'C'))
grade_test_cases.append((65, 'D'))
grade_test_cases.append((50, 'F'))

# Boundary values for each grade range
grade_test_cases.append((100, 'A'))
grade_test_cases.append((90, 'A'))
grade_test_cases.append((89, 'B'))
grade_test_cases.append((80, 'B'))
grade_test_cases.append((79, 'C'))
grade_test_cases.append((70, 'C'))
grade_test_cases.append((69, 'D'))
grade_test_cases.append((60, 'D'))
grade_test_cases.append((59, 'F'))
grade_test_cases.append((0, 'F'))

# Invalid inputs
grade_test_cases.append((-10, None)) # Negative number
grade_test_cases.append((101, None)) # Score above 100
grade_test_cases.append((150, None)) # Score above 100
grade_test_cases.append(("abc", None)) # Non-numeric string
grade_test_cases.append((None, None)) # None input
```



## Subtask:

Implement the `assign_grade` function that takes a score as input and returns the grade according to the specified rules.

[+ Code](#)[+ Text](#)

**Reasoning:** Implement the `assign_grade` function according to the specified rules.



```
def assign_grade(score):  
    """  
    Assigns a grade based on the score.  
  
    Args:  
        score: The score (integer or float) to assign a grade to.  
  
    Returns:  
        The corresponding grade ('A', 'B', 'C', 'D', 'F') or None for invalid scores.  
    """  
    # Check if the score is a valid number and within the range [0, 100]  
    if not isinstance(score, (int, float)) or not (0 <= score <= 100):  
        return None  
  
    # Determine the grade based on the score  
    if 90 <= score <= 100:  
        return 'A'  
    elif 80 <= score <= 89:  
        return 'B'  
    elif 70 <= score <= 79:  
        return 'C'  
    elif 60 <= score <= 69:  
        return 'D'  
    else:  
        return 'F'
```

### Subtask:

Prompt the user to enter a score and validate it using the `assign_grade` function

Reasoning: Prompt the user for input, call the `assign_grade` function with the i

```
user_score_str = input("Enter a score to assign a grade: ")
try:
    user_score = float(user_score_str) # Attempt to convert input to a nu
    assigned_grade = assign_grade(user_score)
    if assigned_grade is not None:
        print(f"A score of {user_score} is assigned a grade of: {assigned_g
    else:
        print(f"Invalid input: '{user_score_str}'. Please enter a number be
except ValueError:
    print(f"Invalid input: '{user_score_str}'. Please enter a numeric val
```

```
→ Enter a score to assign a grade: 80
A score of 80.0 is assigned a grade of: B
```

3• Generate test cases using AI for `is_sentence_palindrome(sentence)`. Ignore case, punctuation, and spaces

Requirement

- Ask AI to create test cases for `is_sentence_palindrome(sentence)` (ignores case, spaces, and punctuation).

- Example:

"A man a plan a canal Panama" → True



## Subtask:

Generate a list of test cases covering various sentences, including palindromes and non-palindromes, punctuation, and spaces.

**Reasoning:** Initialize an empty list for test cases and add tuples of sentences and the specified criteria, including various types of palindromes and non-palindromes.

```
[10] palindrome_test_cases = []

# Simple palindromes
palindrome_test_cases.append(("madam", True))
palindrome_test_cases.append(("racecar", True))

# Palindromes with spaces
palindrome_test_cases.append(("nurses run", True))
palindrome_test_cases.append(("level", True)) # Simple word palindrome

# Palindromes with punctuation and spaces
palindrome_test_cases.append(("A man, a plan, a canal: Panama.", True))
palindrome_test_cases.append(("No 'x' in Nixon", True))

# Palindromes with mixed case
palindrome_test_cases.append(("Racecar", True))
palindrome_test_cases.append(("Madam", True))

# Non-palindromes
palindrome_test_cases.append(("hello world", False))
palindrome_test_cases.append(("python", False))
palindrome_test_cases.append(("not a palindrome", False))

# Edge cases
palindrome_test_cases.append(("", True)) # An empty string can be considered a palindrome
palindrome_test_cases.append((" ", True)) # String with only spaces is a palindrome
palindrome_test_cases.append(("a", True)) # Single character is a palindrome
```

string and comparing it to its reverse.

```
[11] import string

def is_sentence_palindrome(sentence):
    """
    Checks if a sentence is a palindrome, ignoring case, punctuation, and
    Args:
        sentence: The input sentence string.

    Returns:
        True if the sentence is a palindrome, False otherwise.
    """
    # Convert to lowercase
    cleaned_sentence = sentence.lower()

    # Remove spaces and punctuation
    cleaned_sentence = cleaned_sentence.translate(str.maketrans('', '', st

    # Reverse the cleaned string
    reversed_sentence = cleaned_sentence[::-1]

    # Compare the cleaned string with its reversed version
    return cleaned_sentence == reversed_sentence
```

## ✓ Test the function

### Subtask:

Use the generated test cases to test the `is_sentence_palindrome` function and

**Reasoning:** Iterate through the test cases, call the `is_sentence_palindrome` fu

```
[12] for sentence, expected_result in palindrome_test_cases:
      actual_result = is_sentence_palindrome(sentence)
      if actual_result == expected_result:
          print(f"Test passed for sentence: '{sentence}'. Expected: {expected_result}")
      else:
          print(f"Test failed for sentence: '{sentence}'. Expected: {expected_result}")
```

```
⇒ Test passed for sentence: 'madam'. Expected: True, Actual: True
   Test passed for sentence: 'racecar'. Expected: True, Actual: True
   Test passed for sentence: 'nurses run'. Expected: True, Actual: True
   Test passed for sentence: 'level'. Expected: True, Actual: True
   Test passed for sentence: 'A man, a plan, a canal: Panama.'. Expected: True, Actual: True
   Test passed for sentence: 'No 'x' in Nixon'. Expected: True, Actual: True
   Test passed for sentence: 'Racecar'. Expected: True, Actual: True
   Test passed for sentence: 'Madam'. Expected: True, Actual: True
   Test passed for sentence: 'hello world'. Expected: False, Actual: False
   Test passed for sentence: 'python'. Expected: False, Actual: False
   Test passed for sentence: 'not a palindrome'. Expected: False, Actual: False
   Test passed for sentence: '.'. Expected: True, Actual: True
   Test passed for sentence: ', '. Expected: True, Actual: True
   Test passed for sentence: 'a'. Expected: True, Actual: True
```

## ▼ Generate test cases

### Subtask:

Generate a list of test cases covering valid scores within each grade range, boundary values, negative numbers, scores above 100, non-numeric inputs).

**Reasoning:** Initialize an empty list for test cases and add tuples of scores and grades based on the specified criteria.

```
[7] grade_test_cases = []

# Valid scores within each grade range
grade_test_cases.append((95, 'A'))
grade_test_cases.append((85, 'B'))
grade_test_cases.append((75, 'C'))
grade_test_cases.append((65, 'D'))
grade_test_cases.append((50, 'F'))

# Boundary values for each grade range
grade_test_cases.append((100, 'A'))
grade_test_cases.append((90, 'A'))
grade_test_cases.append((89, 'B'))
grade_test_cases.append((80, 'B'))
grade_test_cases.append((79, 'C'))
grade_test_cases.append((70, 'C'))
grade_test_cases.append((69, 'D'))
grade_test_cases.append((60, 'D'))
grade_test_cases.append((59, 'F'))
grade_test_cases.append((0, 'F'))

# Invalid inputs
grade_test_cases.append((-10, None)) # Negative number
grade_test_cases.append((101, None)) # Score above 100
grade_test_cases.append((150, None)) # Score above 100
grade_test_cases.append(("abc", None)) # Non-numeric string
grade_test_cases.append((None, None)) # None input
```

Reasoning: Implement the `assign_grade` function according to the specification.

```
[8] def assign_grade(score):  
    """  
    Assigns a grade based on the score.  
  
    Args:  
        score: The score (integer or float) to assign a grade to.  
  
    Returns:  
        The corresponding grade ('A', 'B', 'C', 'D', 'F') or None for invalid input.  
    """  
    # Check if the score is a valid number and within the range [0, 100]  
    if not isinstance(score, (int, float)) or not (0 <= score <= 100):  
        return None  
  
    # Determine the grade based on the score  
    if 90 <= score <= 100:  
        return 'A'  
    elif 80 <= score <= 89:  
        return 'B'  
    elif 70 <= score <= 79:  
        return 'C'  
    elif 60 <= score <= 69:  
        return 'D'  
    else:  
        return 'F'
```



## Subtask:

Prompt the user to enter a score and validate it using the `assign_grade` function

**Reasoning:** Prompt the user for input, call the `assign_grade` function with the

```
user_score_str = input("Enter a score to assign a grade: ")
try:
    user_score = float(user_score_str) # Attempt to convert input to a float
    assigned_grade = assign_grade(user_score)
    if assigned_grade is not None:
        print(f"A score of {user_score} is assigned a grade of: {assigned_grade}")
    else:
        print(f"Invalid input: '{user_score_str}'. Please enter a number")
except ValueError:
    print(f"Invalid input: '{user_score_str}'. Please enter a numeric value")
```

```
Enter a score to assign a grade: 80
A score of 80.0 is assigned a grade of: B
```